

Capitolo 2 – Stato, logica e change detection nell'era dei Signal

Stato di un componente

Lo stato di un componente è l'insieme dei valori delle sue proprietà in un dato momento. Può cambiare nel tempo, ed è la logica interna del componente a occuparsene.

Interpolazione e metodi nel template

Angular permette di visualizzare sia proprietà dello stato sia risultati di metodi direttamente nel template tramite la sintassi `{{ }}`. Lo sviluppatore dichiara dove deve comparire un valore, e il framework aggiorna la vista automaticamente, senza toccare il DOM manualmente.

Event binding

Per rispondere agli eventi dell'utente Angular usa la sintassi `(nomeEvento)="metodo()"`. Il prefisso "on" viene omissso rispetto al JavaScript classico: `onClick` diventa `(click)`. Il metodo invocato appartiene al componente, non alla pagina.

Il problema della change detection classica

Quando lo stato di un componente cambia, Angular ricalcola indiscriminatamente tutti i binding del template, anche quelli che non dipendono dal valore modificato. Su applicazioni piccole non è un problema, ma su applicazioni reali con centinaia di componenti questo comportamento degrada le prestazioni, la reattività dell'interfaccia e il consumo energetico.

Signal — definizione

Un Signal è una funzione che avvolge un valore di qualsiasi tipo e notifica automaticamente i soggetti interessati quando quel valore cambia. Si dichiara così: `italian = signal(8)`. Nel template va invocato come funzione: `{{ italian() }}`.

Lettura e scrittura di un Signal

`italian()` restituisce il valore corrente. `italian.set(9)` imposta un nuovo valore indipendente dal precedente. `italian.update(v => v + 1)` calcola il nuovo valore a partire da quello corrente ed è preferibile quando il risultato dipende dal valore precedente, perché lavora sempre sull'ultimo valore disponibile ed evita il fenomeno della staleness, ovvero il rischio di sovrascrivere il Signal con un dato già obsoleto.

Computed Signal — definizione

Un computed signal è un Signal il cui valore viene ricalcolato automaticamente, ma solo quando almeno uno dei Signal da cui dipende è cambiato. Si dichiara tramite `computed()` con una arrow function: `techAverage = computed(() => (this.math() + this.programming()) / 2)`. Come tutti i Signal, va invocato nel template come funzione: `{{ techAverage() }}`.

I computed signals sono lazy, ovvero calcolati solo quando vengono letti, e memoized, ovvero il risultato viene memorizzato e riutilizzato finché nessuna dipendenza cambia.

Effect — definizione

Un effect è una reazione automatica al cambiamento di un Signal che non produce un valore di ritorno, ma esegue un'operazione collaterale: una chiamata API, un accesso al `localStorage`, un log. Si dichiara con `effect()` e va inserito nel costruttore del componente, che garantisce il contesto di iniezione necessario ad Angular per gestirne il ciclo di vita. L'effect viene deallocato automaticamente quando il componente viene distrutto. Va usato con attenzione: un utilizzo imprudente può compromettere l'architettura dell'applicazione.